

Parcial de Python — v2

Condicionales · for/while · def & lambda · Ciencia de Datos

⌚ 90 min
📄 20 preguntas
🎯 10 puntos

Instrucciones generales

- ▶ Individual, en computador
- ▶ Solo Python estándar
- ▶ Guarde cada respuesta en un `.py`
- ▶ Sin conexión a internet
- ▶ Cada pregunta vale **0.5 pts**
- ▶ Las preguntas con λ exigen lambda

I CONDICIONALES · PREGUNTAS 1 – 5

- 1 Cree `clasificar_edad(edad)` con las categorías: Niño / Adolescente / Adulto / Adulto mayor.
- 2 Con `notas = [3.2, 4.5, 2.8, 4.9, 3.7, 1.5]`, recorra con `for` e imprima "Aprobó" si ≥ 3.0 , si no "Reprobó".
- 3 Cree `es_correlacion_fuerte(r)` con expresión ternaria: `True` si $|r| \geq 0.7$.
- 4 Con `modelos = {"lineal": 0.85, "arbol": 0.92, "knn": 0.78, "svm": 0.88}`, imprima el de mejor accuracy.
- 5 Cree `es_nulo_o_vacio(valor)`: retorna `True` si es `None`, `""`, `[]` o `{}`.

II CICLOS FOR · PREGUNTAS 6 – 10

- 6 Con `salarios = [2500, 1800, 3200, 4100, 1500]`, halle el promedio y cuántos lo superan.
- 7 Convierta `[88, 92, 79, 95, 85, 70]` a rango 0–1 dividiendo por 100 con un `for`.
- 8 De `clientes = [("Ana", 25), ("Luis", 40), ("Marta", 33), ("Pedro", 19)]`, imprima los nombres de mayores de 30.
- 9 Con `for` anidados, aplane `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` en una sola lista.
- 10 Cuente cuántas palabras de `"machine learning con python"` contienen la letra "a".

III CICLOS WHILE · PREGUNTAS 11 – 15

- 11 Imprima pares del 2 al 20 con `while`.
- 12 Calcule 8! con `while`.
- 13 Cuente los dígitos de `n=12345` sin usar `str`.
- 14 Simule descenso de gradiente: `error=1.0`, multiplique por 0.8 hasta ≤ 0.01 . Cuente iteraciones.
- 15 Recorra `[100, 200, 300, 0, 500, 600]` con `while` sumando hasta encontrar un 0.

λ 16 Escriba `promedio(lista)` con `def` y su equivalente `prom = lambda ...`. Ambas retornan `None` si está vacía.

λ 17 Use `lambda` + `sorted()` para ordenar `empleados` por `salario` descendente.

λ 18 Use `lambda` + `filter()` para extraer impares de `[1,5,8,12,15,20,23]`.

λ 19 Use `lambda` + `map()` para convertir `["1.5", "2.7", "3.9", "4.1"]` a `float`.

λ 20 Cree `crear_clasificador(umbral)` que retorne una `lambda`: `clasif(0.7) → True`, `clasif(0.3) → False`.

Rúbrica y Soluciones — v2

Documento para el docente

🔒 Uso interno



RÚBRICA DE EVALUACIÓN

CRITERIO	PUNTOS
Sintaxis correcta y uso adecuado de estructuras	0.2
Lógica correcta del algoritmo	0.2
Manejo de casos borde	0.1
Total por pregunta	0.5



SOLUCIONES BREVES (REFERENCIA)

```
# 1
def clasificar_edad(edad):
    if edad < 12: return "Niño"
    elif edad < 18: return "Adolescente"
    elif edad < 65: return "Adulto"
    else: return "Adulto mayor"

# 2
notas = [3.2,4.5,2.8,4.9,3.7,1.5]
for n in notas:
    print("Aprobó" if n >= 3.0 else "Reprobó")

# 3
def es_correlacion_fuerte(r):
    return True if abs(r) >= 0.7 else False

# 4
modelos = {"lineal":0.85,"árbol":0.92,"knn":0.78,"svm":0.88}
mejor, val = None, -1
for nom, acc in modelos.items():
    if acc > val: mejor, val = nom, acc
print(mejor, val)

# 5
def es_nulo_o_vacio(v):
    return v is None or v == "" or v == [] or v == {}

# 6
sal = [2500,1800,3200,4100,1500]
p = sum(sal) / len(sal)
c = 0
for s in sal:
    if s > p: c += 1
print(p, c)

# 9
datos = [[1,2,3],[4,5,6],[7,8,9]]
plana = []
for fila in datos:
    for x in fila: plana.append(x)

# 11
i = 2
while i <= 20:
    print(i); i += 2

# 12
n, fact = 8, 1
while n > 1:
    fact *= n; n -= 1
print(fact)

# 13
n, d = 12345, 0
while n > 0:
    n //= 10; d += 1
print(d)

# 14
err, it = 1.0, 0
while err > 0.01:
    err *= 0.8; it += 1
print(it)

# 16
def promedio(lista):
    if not lista: return None
    return sum(lista) / len(lista)

prom = lambda l: None if not l else sum(l) / len(l)

# 17
empleados = [
    {"nombre":"Ana","salario":2500},
    {"nombre":"Luis","salario":3200},
    {"nombre":"Marta","salario":2100},
]
orden = sorted(empleados,
    key=lambda e: e["salario"], reverse=True)

# 18
impares = list(filter(lambda x: x % 2 != 0,
    [1,5,8,12,15,20,23]))

# 19
floats = list(map(lambda x: float(x),
    ["1.5","2.7","3.9","4.1"]))

# 20
def crear_clasificador(umbral):
    return lambda v: v > umbral

clasif = crear_clasificador(0.5)
```

```
print(clasif(0.7)) # True  
print(clasif(0.3)) # False
```